OtakuPulse — Projektdokumentation

OtakuPulse ist die deutschsprachige Anime-Datenbank & News-Plattform unter **www.otakupulse.de**. Diese Dokumentation listet alle wichtigen Pfade mit Erklärungen — als Nachschlagewerk für Betrieb und Weiterentwicklung.

Stand: Juni 2026 · privates, nicht-kommerzielles Projekt.

1. Überblick & Stack

Bereich	Technologie
Framework	Next.js 16 (App Router, React 19)
CMS / ORM	Payload CMS 3.85
Datenbank	PostgreSQL 16
Sprache	TypeScript
Styling	Tailwind CSS 4
KI	Anthropic Claude (Sonnet für News, Haiku für Übersetzung)
Hosting	Docker auf Unraid, Cloudflare-Tunnel (root → www 301)
Analytics	Umami (self-hosted, first-party /stats) + Google Analytics 4 (consent-gated)
Automatisierung	n8n (Social-Autoposter), Telegram-Bot, Host-Cron

Inhaltsstand: ~7.800 Anime, ~29k Charaktere, ~28k Personen, dt. Übersetzungen laufend (KI), Cover lokal gespiegelt.

2. Repository- & Verzeichnisstruktur

Git-Repo unter /mnt/cache/appdata/otakupulse (GitHub: ppschauss/otakupulse).

Pfad	Zweck
frontend/	Next.js + Payload Anwendung (gesamter App-Code)
frontend/src/app/	App-Router-Routen (Seiten, Feeds, API)
frontend/src/collections/	Payload-Collections (Datenmodell)
frontend/src/globals/	Payload-Globals (Navigation, Seiteninhalte, Settings)
frontend/src/lib/	Geteilte Logik & Helfer
frontend/src/components/	React-Komponenten (35 Stück)
frontend/src/scripts/	Import-/Sync-/KI-/Wartungs-Skripte (tsx + Python)
frontend/public/media/	Lokal gespiegelte Cover/Banner (~30k Bilder, Volume)
frontend/secrets.env	Secrets & Envs (gitignored)
docs/	Dokumentation (dieses Dokument, BETRIEB.md, Blueprint)
docker-compose.yml	Stack-Definition (Doku; Container laufen via docker run)
backup-db.sh	Nächtliches DB-Backup + Offsite

Pfad	Zweck
deploy-features.sh	Referenz-Deploy-Skript (Build + Container-Swap)
otakupulse-sync.sh	Cron-Wrapper (täglicher Sync-Lauf)
n8n-workflows/	Versionierter n8n-Autoposter-Workflow + Anleitung

3. Container & Deployment

Container

Container	Rolle	Port
otakupulse-app	Produktions-App (Next start, Image otakupulse:prod, Live-Frontend gemountet)	3001→3000
otakupulse-db	PostgreSQL 16	intern
otakupulse-enrich	Worker: spiegelt Bilder + übersetzt (Loop alle 3 Min); auch Build-/Skript-Runner	—
otakupulse-n8n	Social-Autoposter (RSS → Discord/Telegram/Bluesky/Reddit/Facebook)	5678 (LAN)
umami / umami-db	Self-hosted Analytics, first-party über /stats	intern

Alle laufen via docker run am Netzwerk otakupulse-net (kein Compose-Plugin auf dem Host — docker-compose.yml ist nur Doku).

Deploy-Prozess (Code-Änderungen live schalten)

Die App serviert das kompilierte .next; Code-Edits brauchen Build + Neustart:

1. **Pre-Build** (validiert + erzeugt .next, App bleibt unangetastet bei Fehler): docker exec --env-file frontend/secrets.env -e NODE_ENV=production otakupulse-enrich sh -lc 'cd /app && npm run build'
2. **Container-Swap**: otakupulse-app stoppen/entfernen und mit Image otakupulse:prod, Mount frontend:/app, npm run start, -p 3001:3000, Netz otakupulse-net neu erstellen.

Vorlage: deploy-features.sh (enthält zusätzlich projektspezifische Einmal-Schritte).

Cron (reboot-fest über Unraid)

Cron-Fragmente unter /boot/config/plugins/dynamix/*.cron:

Job	Zeit	Aktion
otakupulse-sync.cron	04:15	otakupulse-sync.sh: AniList-Sync → JustWatch-Streaming → News-Gen → Telegram
otakupulse-telegram.cron	alle 30 Min	telegram-notify (neue Artikel posten)

4. Routen / Seiten (alle Pfade)

App-Router unter frontend/src/app/. Öffentliche Seiten liegen in der Route-Group (frontend), das Admin in (payload).

Kern- & Entitätsseiten

Pfad	Zweck
/	Startseite (Hero, Reihen, Hubs, News, FAQ)
/anime · /anime/[slug]	Anime-Liste & -Detail
/anime/[slug]/episoden · /charaktere	Episoden-/Charakter-Unterseiten
/character · /character/[slug]	Charaktere
/person · /person/[slug]	Personen (Synchronsprecher, Staff)
/studio · /studio/[slug]	Studios (mit Wikidata-Logos)
/genre · /genre/[slug]	Genres
/tag · /tag/[slug]	Tags (nach Kategorie gruppiert)
/season · /season/[slug]	Seasons
/kalender · /kalender/[date]	Simulcast-Kalender
/autor/[slug]	Redaktions-Autorenseiten

Listing & Programmatic SEO

Pfad	Zweck
/suche	Volltextsuche + Filter (Genre, Tag, Sprache, Anbieter …)
/top-anime · /top-anime/[filter]	Bestenlisten (Jahr/Genre)
/vergleich/[slug]	„Anime A vs. B"
/plattform · /plattform/[slug]	Streaming-Anbieter („Wer streamt es?")
/streamen/[slug]	Streaming-Landingpages
/aehnlich/[slug]	Ähnliche Anime
/news · /news/[slug]	News-Liste & Artikel-Detail
/news/kategorie/[category]	Kategorie-Hubs

Community-Landingpages (SEO)

Pfad	Keyword-Fokus
/anime-discord-server	„anime discord server deutsch", „deutscher anime discord"
/anime-telegram-kanal	„anime telegram kanal deutsch", „anime news telegram"

Rechtliches

Pfad	Zweck
/impressum	Impressum
/datenschutz	Datenschutzerklärung (DSGVO)
/nutzungsbedingungen	Nutzungsbedingungen (ToS; u. a. für Google Publisher Center)
/kontakt	Kontaktformular (Collection ContactMessages)

Feeds, Sitemaps & System

Pfad	Zweck
/feed.xml	Haupt-RSS (Teaser + Cover-Bild)
/feed-full.xml	Vollinhalts-RSS (content:encoded, für Flipboard)
/feed/[slug].xml	Kuratierte Themen-Feeds (Collection Feeds)
/genre/[slug]/feed.xml	Genre-Feeds
/news/kategorie/[category]/feed.xml	Kategorie-Feeds
/kalender/feed.xml	Simulcast-Kalender-Feed
/sitemap.xml · /sitemaps/[chunk]	Sitemap-Index + aufgeteilte Sitemaps
/robots.txt · /opengraph-image · /manifest.webmanifest	SEO-/PWA-Basis

Admin & API ((payload))

Pfad	Zweck
/legal/admin	Payload-Admin-Panel (Login p.p.schauss@gmail.com)
/api/[...slug]	Payload REST-API
/api/graphql · /api/graphql-playground	GraphQL

5. Bibliotheken (src/lib)

Datei	Zweck
site.ts	Zentrale Konstanten (URL, Name, DISCORD_INVITE, TELEGRAM_CHANNEL, X_URL)
seo.ts	pageMeta (OG/Twitter/Canonical/Feed-Discovery) + JSON-LD-Builder
feed.ts	RSS-Builder (<enclosure>, media:content, content:encoded)
lexicalHtml.ts	Lexical-Body → HTML-String (für /feed-full.xml)
sitemap.ts	STATIC_PATHS + Chunk-Logik der Sitemaps
anime.ts	Anime-Helfer (Titel, Sprachen, liveArticleFilters …)
anilist.ts · import.ts	AniList-GraphQL-Client + Import-Pipeline
streaming.ts	Anbieter-Register (STREAMING_PROVIDERS, Slugs, Farben)
telegram.ts	Telegram-Versand (Caption-Bau, Foto+Fallback)
hub.ts	SEO-Hub-Aggregation (Top-Genres/Studios/Tags)
navigation.ts · pageContent.ts · settings.ts	Loader für die CMS-Globals
vars.ts · entityVars.ts	Variablen-Ersetzung ({{…}}) in CMS-Texten
genre-info.ts · consent.ts · articleBody.ts	Genre-Texte · Cookie-Consent · KI-Body-Bausteine

6. Skripte (src/scripts) & npm-Befehle

Ausführung im otakupulse-enrich-Container (Live-Code): docker exec --env-file frontend/secrets.env otakupulse-enrich sh -lc 'cd /app && npx tsx …'

npm-Befehl	Skript	Zweck
import:anime	import-anime.ts	Einzelnen Anime von AniList importieren
import:batch	import-batch.ts	Massen-Import (Seiten/Season/Jahr)
import:years · import:tags	…	Jahres-Import · Tag-Kategorien-Backfill
sync	sync.ts	Täglicher Sync (laufende Anime + aktuelle Season + Episoden)
mirror	mirror-images.ts	Cover/Banner lokal spiegeln
translate	translate.ts	KI-Übersetzung (Claude Haiku)
news · news:anime	generate-news.ts · generate-anime-news.ts	KI-News & Anime-Ankündigungen (Claude Sonnet)
telegram:notify	telegram-notify.ts	Neue Artikel in Telegram-Channel posten
seed:navigation	seed-navigation.ts	Navigation-Global initial befüllen
backfill:languages	backfill-languages.ts	Sprach-Felder nachziehen

Python-Skripte (laufen auf dem HOST, curl_cffi):

Skript	Zweck
crunchyroll-dub.py	**Dub/Sub-Sprachen von Crunchyroll (audio/subtitle locales)**
justwatch-streaming.py	**Streaming-Anbieter-Breite (JustWatch DE)**
wikipedia-dub.py	**Deutsche Dubs aus Wikipedia/Wikidata**
studio-logos.py	**Studio-Logos & Websites aus Wikidata/Commons**

7. Payload-Collections & Globals

Collections (src/collections): Anime, Episodes, Characters, Persons, Studios, Genres, Tags, AnimeSeasons, Platforms, Articles, Authors, Feeds, ContactMessages, Users.

Globals (src/globals): Navigation (Header/Footer-Links), PageContent (pflegbare Meta-/Footer-Texte je Übersichtsseite), Settings.

Hinweis: Die Footer-„Community"-Spalte (Discord/Telegram/X-Landingpages) ist **fest in components/Footer.tsx** verdrahtet, nicht im CMS.

8. Komponenten (Auswahl)

Komponente	Zweck
Header.tsx · Footer.tsx	**Navigation inkl. Discord/Telegram/X-Buttons**
RichText.tsx	**Lexical-Renderer (React)**
SubscribeWithGoogle.tsx	**Google „Subscribe with Google" (nur Artikelseiten)**
Faq.tsx	**FAQ-Akkordeon + FAQPage-Schema**
Breadcrumbs.tsx · JsonLd.tsx	**Breadcrumbs + Structured Data**
HubSections.tsx · SeoFooter.tsx	**SEO-Hub-Bausteine**
MediaRow/ScrollRow/AnimeCard	**Crunchyroll-Stil-Carousels & Karten**

9. RSS-Feeds (Übersicht)

Öffentliche Übersicht: /feeds. Alle Feeds tragen Cover-Bilder (<enclosure> + media:content). Jede Seite weist den Haupt-Feed via <link rel="alternate"> aus (Auto-Discovery).

- https://www.otakupulse.de/feed.xml — Haupt-Feed (Teaser)
- https://www.otakupulse.de/feed-full.xml — Vollinhalt (Flipboard)
- …/kalender/feed.xml — Simulcast-Kalender
- …/news/kategorie/<cat>/feed.xml — 7 Kategorien
- …/genre/<slug>/feed.xml — 19 Genres
- …/feed/<slug>.xml — kuratierte Themen-Feeds

10. Social & Community

Kanal	Verlinkung / Mechanik
Discord	**DISCORD_INVITE — Header/Footer/Homepage + Landingpage /anime-discord-server**
Telegram	**TELEGRAM_CHANNEL (@otakupulse_de) — Bot postet Artikel automatisch; Landingpage /anime-telegram-kanal**
X / Twitter	**X_URL (@OtakuPulseDE) — Header/Footer**

n8n-Autoposter (n8n-workflows/otakupulse-autoposter.json): zieht intern http://otakupulse-app:3000/feed.xml und postet neue Artikel nach Discord, Mastodon, Bluesky, Reddit und Facebook. Dedup über den RSS-Trigger. Tokens trägt der Betreiber im n8n-UI ein (http://<host>:5678).

11. SEO

Baustein	Ort
Metadaten (OG/Twitter/Canonical/Feed-Discovery)	**lib/seo.ts → pageMeta**
OG-Bild (generiert)	**app/(frontend)/opengraph-image.tsx**
Structured Data	**lib/seo.ts (Organization, Breadcrumb, Anime/TVSeries, Person) + NewsArticle in news/[slug]**
Sitemaps	**app/sitemap.xml + app/sitemaps/[chunk] + lib/sitemap.ts**
FAQ-Schema	**components/Faq.tsx**

Baustein	Ort
Google „Subscribe with Google"	**components/SubscribeWithGoogle.tsx (Publication CAow_dDLDA, Open Access)**

12. Sicherheit

Header in frontend/next.config.mjs (öffentliche Routen, Admin ausgenommen): HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, **COOP** (same-origin), **CORP** (cross-origin), **COEP** (credentialless), strenge **CSP** (inkl. Freigaben für GA und news.google.com), poweredByHeader: false.

Weiteres: public/.well-known/security.txt (RFC 9116). Offen: DNSSEC (manueller Cloudflare-Schritt).

13. Backups & Secrets

Backups (backup-db.sh, nächtlicher Cron): pg_dump von App- + Umami-DB → /mnt/cache/appdata/backups, gzip, Rotation 14 Tage, Offsite nach Backblaze B2 via rclone (Remote offsite).

Secrets (frontend/secrets.env, gitignored): DATABASE_URI, PAYLOAD_SECRET, ANTHROPIC_API_KEY, TELEGRAM_BOT_TOKEN, TELEGRAM_CHANNEL_ID, N8N_ENCRYPTION_KEY, NEXT_PUBLIC_* (Site-URL, Umami, GA-ID).

14. Datenquellen & Enrichment

- **AniList** (GraphQL) — Stammdaten (rate-limitiert ~28/min); On-Demand-Fallback beim ersten Aufruf unbekannter Slugs.
- **Crunchyroll / JustWatch / Wikipedia / Wikidata** — Dub/Sub-Sprachen, Streaming-Anbieter, Studio-Logos (Host-Python-Skripte, idempotent, wöchentlich).
- **Claude (Anthropic)** — deutsche Übersetzungen (Haiku) und News-Texte (Sonnet).
- Bilder werden lokal nach public/media gespiegelt (Worker otakupulse-enrich).

15. Offene Punkte / Roadmap

- **DNSSEC** in Cloudflare aktivieren (DS-Record beim Registrar).
- **n8n-Tokens** eintragen (Discord/Telegram sofort, dann Reddit-Sub/Facebook-Review).
- **Flipboard**-Einreichung mit /feed-full.xml.
- Optional: WebSub/PubSubHubbub-Hub + Google-News-Sitemap für Echtzeit-Verbreitung.
- Secrets-Rotation (ANTHROPIC_API_KEY, TELEGRAM_BOT_TOKEN) nach früherem Leak.

Generiert für OtakuPulse — interne Projektdokumentation.